



<https://www.linkedin.com/company/quantum-robotics-automation-research-group/?viewAsMember=true>

Research Innovation and Technical Writing CSE414 (Section: 65 C/E)

Lecture-8

Instructor

***Project Management & Financial Analysis
CEP & CEA***

Dr. M. Akhtaruzzaman

Associate Professor and Head, RME, DIU

akhtaruzzaman.cse@diu.edu.bd; akhter900@gmail.com; akhter900@qrarg.com

July 7, 2026, DIU, Savar, Dhaka.

رب زدني علما وارزقني فهما.

My Lord! Advance me in Knowledge and True Understanding.

Project Management & Financial Analysis

Knowledge Profile + CEP/CEA Justification + Abstract/Methodology + Future/Limitation + IEEE Citations

Running Example

Multimodal Fake News Detection with Explainable AI
(Text + Image) for Social Media Posts

Deliverables

- Abstract (100–300 words)
- Methodology / System Design + Requirements
- Project Plan (scope → WBS → timeline)
- Task Allocation
- Budget + Rationale
- CEP mapping + rationale
- Knowledge Profile mapping (linked to EP1) + rationale
- CEA mapping + rationale
- Limitations + Future Work
- IEEE citations + references

Abstract

What an Abstract must do:

- One paragraph, typically 100–400 words
- Covers: background/problem → methodology → (Phase-I) planned evaluation/results → implication
- Every sentence adds information (no filler)

Abstract formula

- 1) Problem (1–2 lines): “This project addresses ... because ...”
- 2) Approach (2–3 lines): “We propose ... using ... (model + data + key idea)”
- 3) Evidence (1–2 lines): “We evaluate using ... metrics (e.g., F1) and baselines (text-only/image-only).”
- 4) Why it matters (1–2 lines): “This helps ... by ... while supporting responsible use via explainability.”

Abstract

Example abstract (≈150–180 words):

Social media misinformation spreads through both text and images, making manual verification slow and inconsistent. This project proposes a multimodal fake news detection system that classifies a post as likely fake or likely real using both its caption text and attached image. We design a pipeline that cleans and tokenizes text, normalizes images, and trains three models: (i) a text-only baseline, (ii) an image-only baseline, and (iii) a multimodal fusion model. To support trustworthy use, the system provides explanations by highlighting influential words and visual regions that contributed to the prediction. We plan to evaluate the models with precision, recall, and F1-score and compare accuracy-vs-latency trade-offs for deployment in a lightweight web demo. The expected outcome is an explainable prototype that demonstrates how multimodal learning can improve robustness compared to unimodal baselines while encouraging responsible interpretation of automated predictions.

Chapter 3: Methodology vs System Design (don't mix them)

Methodology

Template rule of thumb:



Methodology (research thinking)

Explains HOW you investigate/validate.

For ML/DL: data → modeling choices → training → evaluation protocol.

Focus: experimental logic, fairness of comparison, metrics.



System Design (engineering build)

Explains WHAT you are building and HOW components interact.

Architecture: UI → API → model service → database.

Focus: modules, interfaces, data flow, deployment constraints.

Beginner mistake (and fix)

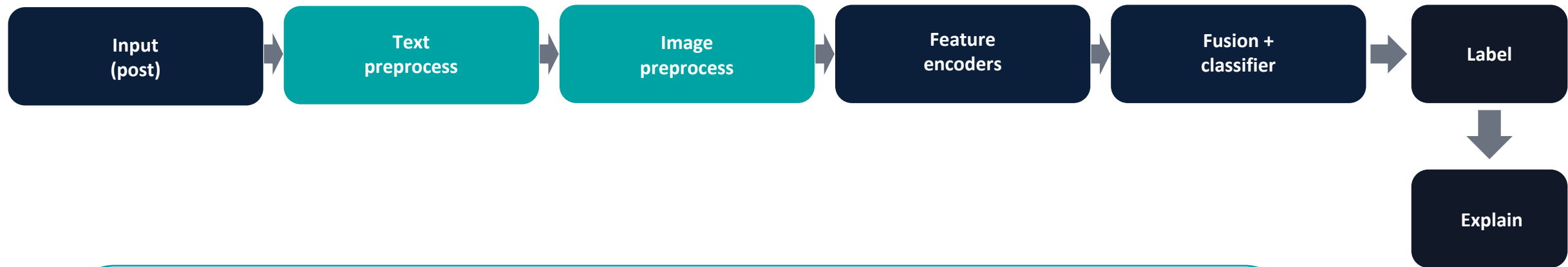
Mistake: writing “We used CNN + BERT” but not explaining the pipeline.

Fix: write 5–7 steps in order (inputs → preprocessing → models → fusion → outputs → metrics).

Example pipeline (draw this as Figure 3.x)

Methodology

Multimodal pipeline (text + image) → label + explanation



What to write under “Detailed Methodology & Design”

- Datasets + cleaning rules (what you remove and why)
- Baselines (text-only, image-only) + your multimodal model
- Evaluation protocol (train/val/test split, metrics, number of runs)
- Explainability method (what the explanation shows + limitations)

Project mgmt

Good plans are about outputs, not “we will work hard”.



Milestones

M1 Dataset + preprocessing pipeline
M2 Baselines (text-only, image-only)
M3 Multimodal fusion model
M4 Explainability module
M5 Demo app (API + UI)
M6 Evaluation + report writing

Tip: each milestone must have a clear artifact (code, dataset, figure, table).



Task allocation

Student A: data + baselines + evaluation
Student B: multimodal model + explainability

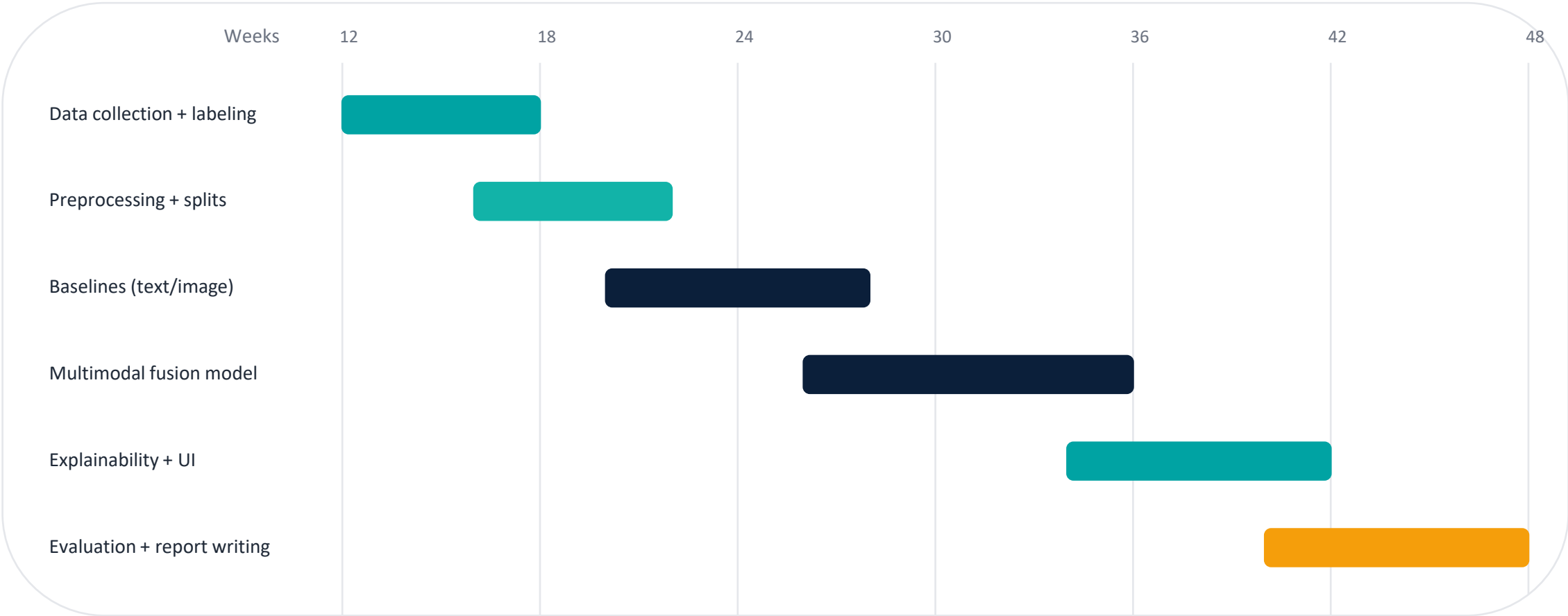
Weekly ritual:

- 1 goal per person
- 1 risk per person
- 1 deliverable per sprint

Task allocation timeline (Weeks 12–48) — example

Project mgmt

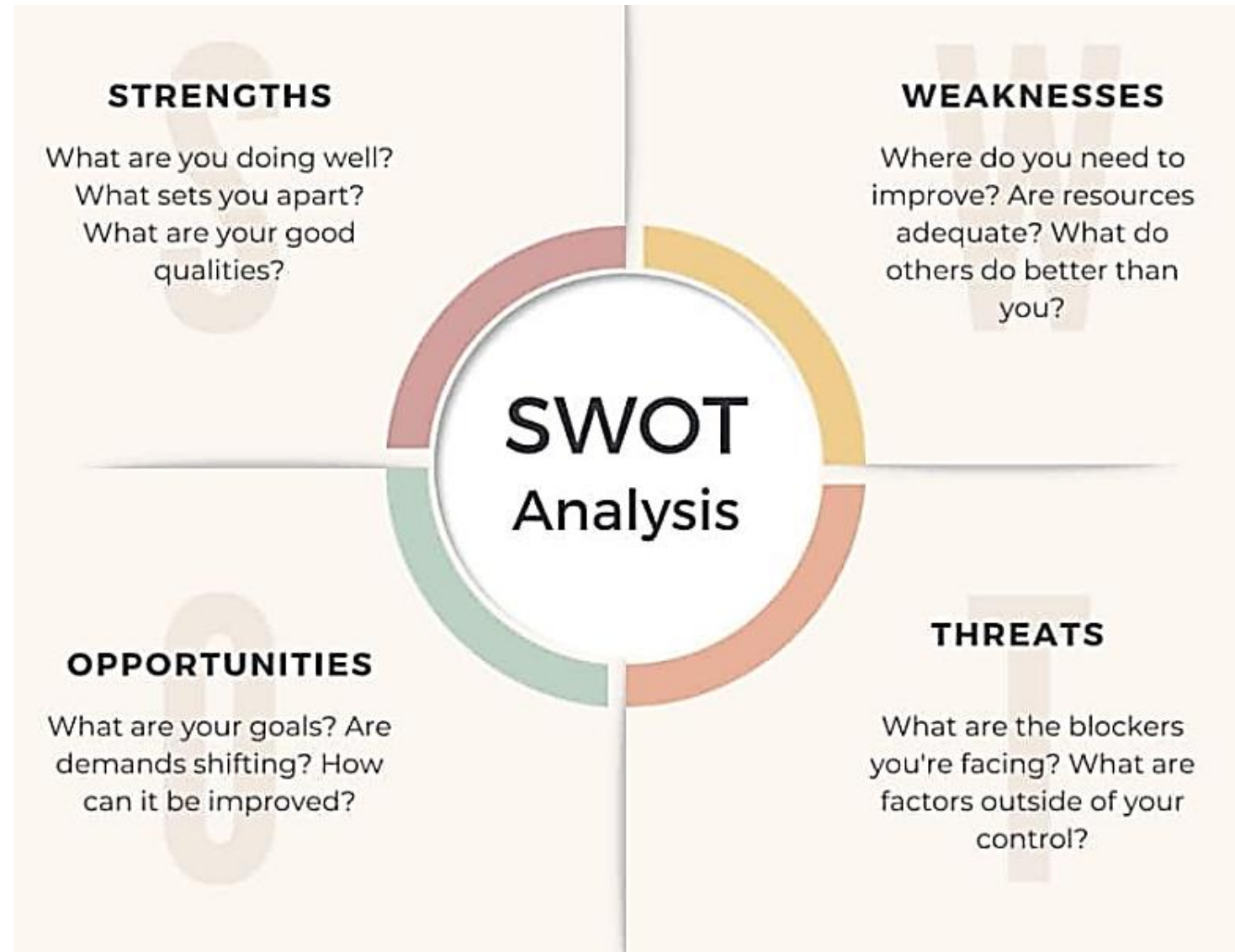
Use the template’s “weeks table” but fill it with YOUR real phases.



Tip Keep phases overlapping slightly (real projects overlap: data ↔ modeling ↔ app).

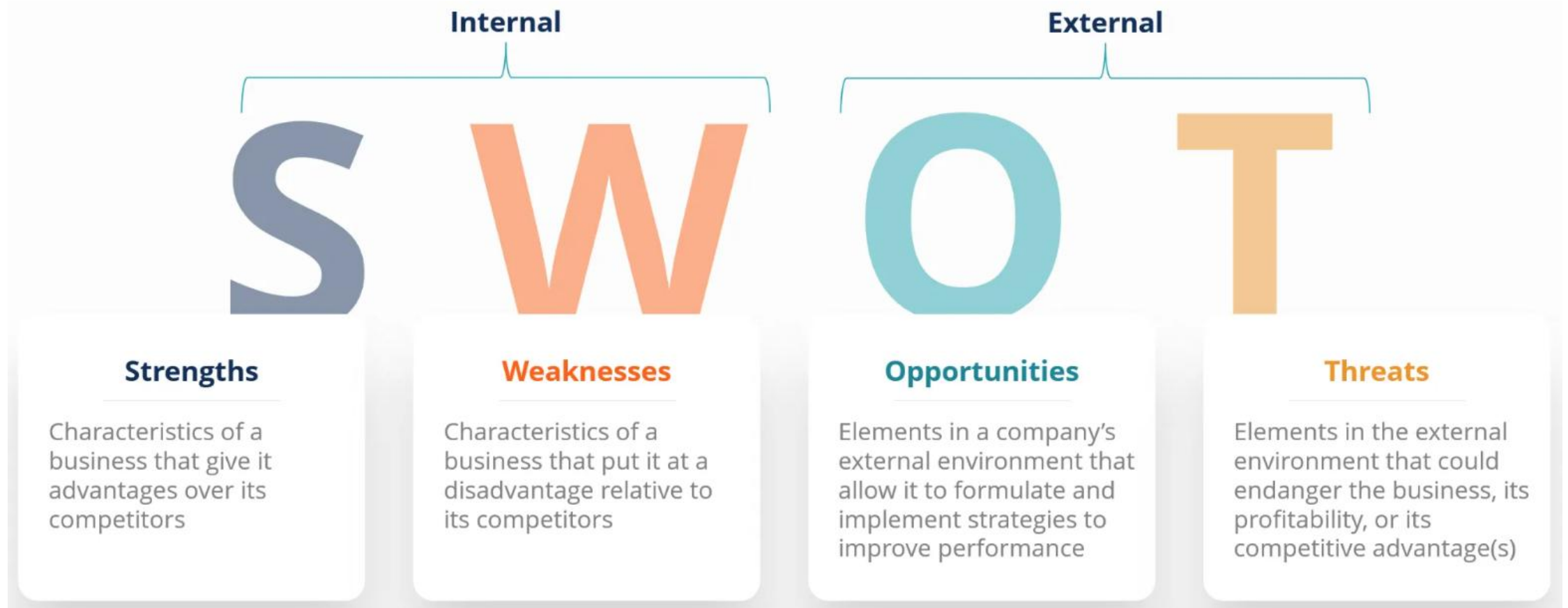
Add a small risk table to prove you thought like an engineer.

Risk	Prob.	Impact	Mitigation (backup plan)
Dataset is noisy / labels unreliable	M	H	Use distant supervision + manual review on a small validation set; report limitations.
Model overfits (high train score, low test score)	M	M	Use regularization, early stopping, stronger baselines; keep test set untouched.
Deployment latency too slow for demo	H	M	Use smaller model, batch inference, caching; show “lite” mode in demo.
Misuse concerns (tool used to censor)	L	H	Add disclaimers, explainability, and “human-in-the-loop” review recommendation.



SWOT ANALYSIS





Write a statement why your solution will be applicable over these weakness and Threats

Financial Analysis (5.3): what you MUST include

Finance

Write these 3 things

- 1) Budget (cost analysis)
- 2) Revenue model (how it could earn / justify value)
- 3) Alternate budget + rationales (minimum vs recommended)



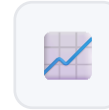
Budget = list items + why needed

Format (simple):

Item | Unit cost | Qty | Total | Why

Include ML costs too:

- GPU hours
- storage
- hosting
- APIs
- backups



Revenue model = who pays + why

Pick ONE model and justify:

- Subscription (SaaS)
- One-time license
- Freemium
- B2B integration

For FYDP: keep it realistic and small-scale.

Finance

Budget categories you should consider:

- Compute: GPU/CPU time (training + testing) + electricity (optional estimate)
- Hosting: backend server + database + object storage
- Domain & security: domain, SSL, backups, monitoring
- Data: collection tools, labeling time (even if “free”, mention effort)
- APIs: e.g., email/SMS for login, image processing services (if used)
- Hardware (optional): camera/sensors if your project uses IoT

Two-budget method (recommended)

Budget A (Minimum viable): free tiers + small compute + basic hosting.

Budget B (Recommended): paid hosting + more compute + security + backups.

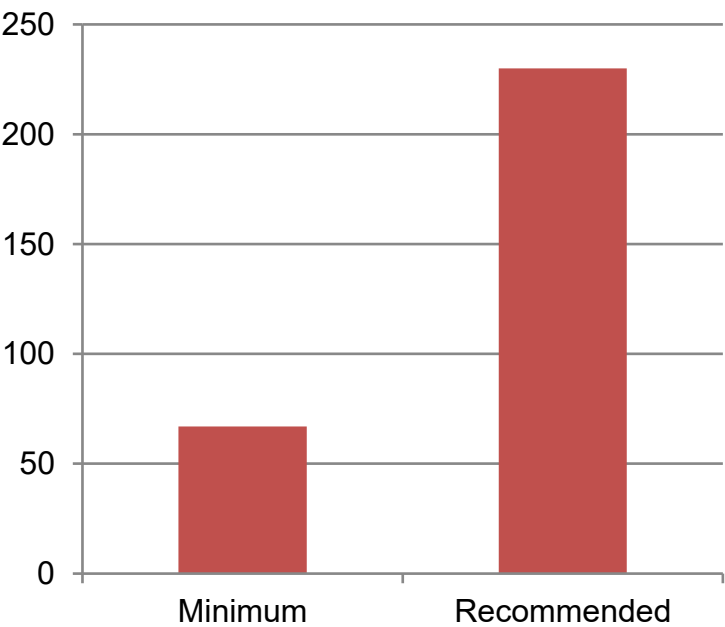
Budget example (Minimum vs Recommended)

Finance

Example numbers

Item	Min (USD)	Rec (USD)	Rationale
Cloud hosting	\$10	\$25	API + demo uptime
GPU compute	\$30	\$120	train & tune models
Storage/DB	\$5	\$15	datasets + logs
Domain + SSL	\$12	\$20	trust + HTTPS
Monitoring/Backup	\$0	\$20	reliability
Misc (labels/tools)	\$10	\$30	label fixes
Total	\$67	\$230	

Total cost comparison



How to justify alternate budget

Explain what breaks if you use Minimum (e.g., fewer runs → lower accuracy).

What is an Engineering Problem?

What CEP answers

An **engineering problem** is a situation where:

Something needs to be **designed, improved, or fixed**

We apply **science, mathematics, and engineering knowledge**

The goal is to find a **practical and safe solution**



Example:

Designing an algorithm to sort large amounts of data faster, Creating a mobile app to manage online classes, Detecting spam emails using machine learning

Engineering problems are generally classified into:

- **Simple Engineering Problems**
- **Complex Engineering Problems (CEP)**

A Complex Engineering Problem usually involves(informal):

- ✓ **Multiple and conflicting requirements**
- ✓ **Uncertain or incomplete information**
- ✓ **Interaction of many systems**
- ✓ **Need for innovation and judgment**
- ✓ **Consideration of social, economic, and environmental impacts**

Engineering Problem: EP1–EP7 in plain language

EP

EP1

Depth of knowledge

Needs more than basic coding (ML + vision + NLP + deployment)

EP2

Conflicting requirements

Improving one metric may hurt another (accuracy vs speed vs privacy)

EP3

Depth of analysis

Requires careful evaluation, ablations, and error analysis

EP4

Familiarity of issues

Not a copy-paste tutorial; data shifts and ambiguity exist

EP5

Applicable codes/standards

Must follow standards (security, privacy, citation, APIs)

EP6

Stakeholder involvement

Multiple stakeholders with different needs/risks

EP7

Interdependence

Many modules depend on each other (pipeline ↔ model ↔ app)

CEP

What CEP answers

“Is the PROBLEM complex enough to be considered an engineering design project?”
You prove this by mapping your project to EP1–EP7 and writing short rationales.

What is CEP?

A **Complex Engineering Problem (CEP)** is a problem that:

- **Cannot be solved easily**
- Has **multiple constraints**
- Requires **advanced engineering knowledge**
- Often has **no single correct solution**

✚ These problems exist in **real-world engineering practice**

*A **Complex Engineering Problem (CEP)** is an engineering problem that **necessarily requires in-depth engineering knowledge** and **additionally exhibits one or more recognized complexity attributes** related to **uncertainty, scale, impact, or system interaction**.*

What is CEP?

Mandatory Condition (EP1)

● EP1 – Depth of Engineering Knowledge (Compulsory)

The problem **cannot be solved using routine methods** and **requires in-depth engineering knowledge** at the level of:

Advanced engineering fundamentals

Engineering design or practice

Research-based or specialized knowledge

✚ *Without EP1, a problem cannot be classified as CEP.*

At least **one or more** of the EP2-EP7 must be present:

Logical Rule for CEP Classification

$$\text{CEP} = \text{EP1 (Mandatory)} + (\text{Any one or more of EP2–EP7})$$

CEP

Write EP level + evidence (short, specific).

EP	Attained	Justification(1–2 lines)
EP1	Yes	Requires NLP + CV + multimodal fusion + explainability + deployment.
EP2	Yes	Accuracy vs latency vs privacy (storing images/logs) must be balanced.
EP3	Yes	Needs baselines + ablations + robustness/error analysis (text-only vs image-only vs fusion).
EP4	No	
EP5	No	
EP6	No	
EP7	Yes	Pipeline interdependence: data → model → explainability → API → UI → deployment.

Must be justified with proper explanation

Knowledge Profile (K1–K8): show what knowledge you used

Knowledge

What Knowledge Profile answers

“Which knowledge areas are required to solve this problem?”
In Phase-I, you must map the overall problem and EP1 to K1–K8 and write rationale.

Template hint
To justify EP1, use multiple among K3, K4, K5, K6, K8 (as the template suggests).

K1	Natural science	Often not central in CSE (unless sensors/physics)
K2	Mathematics	Stats, probability, linear algebra
K3	Engineering fundamentals	DSA, DB, OS, networking
K4	Specialist knowledge	ML, NLP, CV, XAI
K5	Engineering design	architecture + trade-offs
K6	Engineering practice	tools: Git, testing, deployment
K7	Comprehension	understanding real context + stakeholders
K8	Research literature	papers, baselines, gap analysis

Knowledge Profile (K1–K8): show what knowledge you used

Knowledge Profile (FYDP Report | Section 5.4.1)

A **Knowledge Profile** defines the **level and depth of knowledge** required to **understand, analyze, and solve an engineering problem**.

👉 It indicates **how advanced the knowledge must be**, ranging from basic sciences to research-based engineering knowledge.

*The **Knowledge Profile (K1–K8)** was defined by the **International Engineering Alliance (IEA)** through the **Washington Accord**.*

Levels of Engineering Knowledge (K1–K8)

- **K1 – Mathematics & Basic Sciences**
Fundamental mathematics, physics, and chemistry
- **K2 – Engineering Fundamentals**
Core engineering concepts applicable across disciplines
- **K3 – Specialist Engineering Knowledge**
Discipline-specific engineering knowledge
- **K4 – Advanced Engineering Fundamentals**
Deeper theoretical understanding and advanced analysis
- **K5 – Engineering Design Knowledge**
Knowledge required to design systems, components, or processes
- **K6 – Engineering Practice Knowledge**
Practical knowledge used in real-world engineering applications
- **K8 – Research Literature**
Knowledge derived from current research and scholarly sources

Knowledge Profile and Problem Complexity

- **Simple Engineering Problems** typically require **K1–K2**
- **Complex Engineering Problems (CEP)** require **K3 or higher**

• **EP1 (mandatory for CEP)** is satisfied when **K3 / K4 / K5 / K6 / K8** knowledge is required

📌 Higher knowledge levels indicate **greater depth, analysis, and complexity**.

Knowledge Profile mapping (example filled)

Knowledge

Example mapping for “Multimodal Fake News Detection + XAI”

K	Used?	Justification
K1	No	No physics/chemistry domain required (pure software + data).
K2	Yes	Statistics for metrics (precision/recall/F1), loss functions, confidence.
K3	Yes	Data structures, databases, APIs, and system integration.
K4	Yes	Specialist ML: transformers/CNNs, multimodal fusion, explainability.
K5	Yes	Architecture trade-offs: accuracy vs latency vs privacy.
K6	Yes	Engineering practice: Git, experiment tracking, testing, deployment.
K7	Yes	Stakeholder context: moderation workflows + misuse risk.
K8	Yes	Literature review: baselines, dataset choice, gap analysis.

EA

Engineering Activity

An **Engineering Activity (EA)** refers to the **professional actions and tasks performed by engineers** to apply engineering knowledge in **designing, developing, implementing, operating, or improving engineering systems, products, or processes**.

Who Proposed the Concept of EA?

The concept of **Engineering Activities (EA)** was defined by the **International Engineering Alliance (IEA)** under the **Washington Accord**, as part of the **Graduate Attributes and Professional Competencies (GAPC)** framework.

Relationship Between EA, EP, CEP, and Knowledge Profile

Concept	Focus
CEP (Complex Engineering Problem)	Complexity of the problem
EA (Engineering Activity)	Actions performed by engineers
Knowledge Profile (K1–K8)	Depth of knowledge applied

- 👉 CEP focuses on “what is being solved”
- 👉 EA focuses on “what is being done”

EA

EA1

Range of resources

Uses multiple resources (datasets, GPU, cloud, APIs, tools)

EA2

Level of interaction

Integrations + real user/system interactions (UI ↔ API ↔ DB ↔ model)

EA3

Innovation

A clear new element (method/pipeline/feature) beyond copying a tutorial

EA4

Consequences

You analyze societal/environment impacts and design mitigations

EA5

Familiarity

Handles unknowns: noise, edge cases, deployment constraints, failures

CEA

Example mapping: activities + evidence

EA	Evidence (what we did)
EA1	Used multimodal dataset + GPU training + cloud hosting + libraries (PyTorch, Transformers) + experiment tracking.
EA2	Built web demo: UI → API → model inference service → database; added admin dashboard for metrics.
EA3	Designed fusion model and explanation layer combining text highlights + image saliency for transparency.
EA4	Analyzed societal risks (misinformation, bias, misuse). Proposed mitigations: disclaimers, human-review workflow, minimal logging.
EA5	Handled non-routine issues: noisy labels, sarcasm, domain shift, latency constraints; iterated with error analysis.

Writing

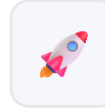


Limitations (what you could NOT do now)

Write the “why” clearly:

- Limited labeled data → weak generalization
- No real deployment test → only offline evaluation
- Explainability is approximate → may be misleading

Good sentence: “Due to X, we could not Y, so Z remains untested.”



Future work (what you will do next)

Concrete and measurable:

- Add robustness tests (domain shift, adversarial images)
- Optimize latency (quantization / distillation)
- Human-in-the-loop feedback loop

Good sentence: “Next, we will implement X and measure Y on dataset Z.”

IEEE referencing (minimal rules students must follow)

- Cite in text with numbers in brackets: “... as shown in [1].”
- Reference list is numbered in order of first appearance (not alphabetical).
- Cite datasets, tools/libraries, baselines, and key claims.

[1] H. Author, “Paper title,”
Journal, vol., no., pp., year.

In-text example: “... improves
robustness [1].”

Wrap-up

If you can tick these boxes, your report becomes “reviewer-proof”.

- ✓ Abstract: includes problem + method + evaluation plan + implication; 100–300 words.
- ✓ Methodology: pipeline steps are clear; includes baselines and metrics.
- ✓ Requirements: functional + non-functional are measurable.
- ✓ Finance: budget + revenue model.
- ✓ CEP: EP1–EP7 mapped with evidence (not generic).
- ✓ Knowledge Profile: K1–K8 mapped; EP1 supported using K3/K4/K5/K6/K8.
- ✓ CEA: EA1–EA5 mapped with artifact evidence.
- ✓ References: IEEE style, and you cite datasets/tools/baselines.

Life is, you verses you, a zero-sum game.

Thank You